SMU Classification: Restricted

IS216 WAD2

Project

Rubrics

Progress Pitch (5 marks)

Component *(Check with your faculty/instructor how the progress pitching will be conducted)*		%
	Clearly defined problems & features	25%
	Clearly defined web design & tech stack	25%
	Clearly defined job scopes / task delegation	25%
	A functioning task – e.g., backend service / API call / CRUD operation – a feature or a sub-feature	25%

Final Presentation and Deliverables (20 marks)

Component		%
(1) Presentation		**25%**
	The presentation	20%
	Q & A	5%
(2) Working Web Application		**75%**
	Problem(s) solving and solution appropriateness	20%
	Working app, correctly implemented and usable solution	27%
	Styling (including responsive design)	18%
	Testing	10%

Peer evaluation (5 marks)

Each team member will submit the peer evaluation form, which will be evaluated based on the teaching team's professional judgement. For any conflict, the team should first try to manage and resolve internally rather than simply waiting it out and stating the conflicts in the peer evaluation form. If a member is difficult to manage, report to the teaching team early (**at least 3 weeks before the deadline**) so that the teaching team can intervene and take action. If the teaching team judges that the team did not make such an effort to manage conflicts, all the team members may still get the same grade, regardless of peer evaluation scores. Teamwork and project management is an implicit part of the project grading.

In an exceptional situation where a member is not contributing anything significant or s/he is missing majority of the team meetings/discussions, the member will be penalized depending on the severity, including the possibility of zero mark for the entire project.

SMU Classification: Restricted

Rough Guide

- Up to B+/A- :
  - CSS - aesthetically pleasing, responsive, logical layout
  - JS - intuitive, interactive, animated UIs
  - JS/Backend - API/data processing and functionality implementations
  - (B+ : get data -> show data, A- : get data -> merge data -> massage data -> show data)
  - Can be self-built backend OR (encouraged) free & open public APIs

- X-factor (A/A+ -- solving a challenging business problem that requires logics/algorithm
  - E.g. coming up with a better algorithm for BOSS bidding, a pricing strategy for an e-commerce platform based on buyers' behaviors, course mapping algorithm for overseas exchange program, stock data analysis and recommendation …

- Incorporate knowledge and skills from IDP (UI design) and Computation Thinking (logics/algorithm)

- Demonstrate Interpersonal skills – team/project management, communication, presentation

SMU Classification: Restricted

What topic should we choose?

We are very flexible. Any topic is fine – what matters is the problem and the details, not the theme. We are deliberately flexible on topics.

What we look for is that your app:

- Solves a specific, well defined problem with a clear target of user group
    - Weak: "A travel website" (Who is it for? What problem does it solve?)
    - Strong: "An app that helps exchange students plan multi-city trips within a budget by combining flight, hostel and event data"
- Is a realistic web app – something that can be deployed and used by real people (not just a collection of pages)
- Popular past themes include Travel / Events management system, Study group, Food / Restaurant / Volunteers finder, Food recipes, Movie reviews / recommendations, Escape room , Language learning
- Whatever you pick, be able to answer in one sentence: "Our app helps [who] to [do what] better because [how]
- **Tip:** Topics where real data exists (free public APIs) are easier to make impressive, because marks come from *processing* interesting data with some logics / algorithms, not just displaying it.

SMU Classification: Restricted

What kind of features should we build?

Think of features in three tiers, which map directly to the grading guide:

- **Tier 1** — Get data, show data (up to ~B+): Fetch data from an API or your backend and display it in a responsive, aesthetically pleasing, interactive UI. This demonstrates competence but is the baseline.
- **Tier 2** — Get data, merge/massage data, show data (~A-): Combine multiple data sources, filter, aggregate, transform, or compute something before presenting it. Example: not just listing restaurants, but ranking them by combining distance, dietary filters, and group preferences.
- **Tier 3** — X-factor: logic and algorithms (A/A+): Solve a genuinely challenging business problem that requires you to design logic or an algorithm — e.g., a smarter bidding strategy, a matching or recommendation engine, a pricing or scheduling algorithm. This is where you apply what you learned in Computational Thinking, on top of the UI/UX skills from IDP.

A good project has a mix: solid Tier 1/2 features forming the core app, plus at least one Tier 3 feature that is your team's "signature".

What does NOT count as a feature: login/logout and user registration. With Supabase or Firebase these are near plug-and-play, so treat them as plumbing, not a feature. (And do not demo them in the final presentation.)

What exactly count as a feature?

- A common mistake is ***counting sub-tasks*** as features.
  - A sub-task is just one step inside that journey. The litmus test: "Would a user tell a friend: 'this app lets me ___'?" If the blank is "click a button to add an item to a list," it's a sub-task. If it's "plan my whole trip day by day," it's a feature.

  - For example, in our ***mini shopping-cart code along exercise*** that we did in Week 6,
    - "displaying items for the selected category" "adding an item to the cart" **are sub-tasks.**
    - "displaying items for the selected category" -> "adding an item to the cart" -> "Check out the cart" / "Reset the cart" **is a feature**

- The rule: **A feature** lets a user complete one distinct, meaningful task end-to-end — from input, through processing, to a visible outcome. A sub-task is just one step inside that journey.

A Worked Example : Travel Management App

- ONE feature: "Group Itinerary Planner" — plan a multi-day trip collaboratively and get a conflict-free, budget-aware itinerary.
- This feature is made up of many sub-tasks. None of them is a feature on its own:
  - Search for attractions via a public API (e.g., Google Places API)
  - Display results on a list and map view (Google JavaScript Maps)
  - Add an attraction to a day slot in the itinerary
  - Drag-and-drop to reorder items within/between days
  - Compute total daily cost against the group's budget
  - Flag conflicts (overlapping timings, opening hours, travel time between spots)
- Six pieces of work — but one feature, because the user's task is "plan my trip," and the task is only complete when all steps work together.
- The classic inflation mistake: counting "search page," "results page," and "detail page" as three features. Those are three screens serving one task ("find an attraction") — that is one feature at most.
- Three distinct features for the same travel app, mapped to tiers:

Feature	Why it's a feature	Tier
Browse and explore destinations with live weather, photos, and events	Task: "discover where to go"	Tier 1-2
Group Itinerary Planner (above)	Task: "plan the trip"	Tier 2
Cost-splitting engine: log shared expenses in mixed currencies, compute who-owes-whom with the fewest transfers	Task: "budget sharing fairly"; requires a real algorithm (debt simplification, live FX rates)	Tier 2-3

How many features?

- Rough guide: one meaningful feature per person — so about six for a 6-member team.
- But note:
  - This is a guide, not a quota. One well-designed algorithmic feature (Tier 3) is worth more than three shallow CRUD screens. We grade depth of problem-solving, not the length of your feature list.
  - Not everyone must "own a feature". Sensible role division counts —
    - e.g., one member owns backend and database, another owns UI/UX consistency across all pages. What matters is that every member makes a significant, visible contribution, and that the total scope reflects six people's work over the term.
- Ensure that no two persons is responsible for the same thing, i.e., "Alice & Bob are responsible for creating the database." -> Have distinct role and responsibilities.

SMU Classification: Restricted

What Tech Stack to use?

- Any backend / frontend technology is fine. Essentially you are to demonstrate the concepts / skills learnt / taught in WAD2 and other relevant courses -> build an app that is problem solving, responsive, interactive, reactive, intuitive, modular
- The usual choice of tech stack include :
  - Database : Firebase / Supabase / MongoDB
  - Backend : Express.js / Python Flask
  - Frontend : CSS, Bootstrap / Tailwind, Vue.js / React
  - Deployment (optional) : Vercel / AWS / Heroku / Firebase
  - Version control : Github

SMU Classification: Restricted

Project Progress Evaluation

- We expect you to make some progress by the end of recess week
- Progress evaluation is on Week 9 Monday
  - There is no lecture on Week 9. There is no class.
  - Send us your progress + progress tracking report by Week 9 Monday 23:59PM, for our evaluation.
- What you need to do:
  - Write up the project proposal using the template provided in **eLearn -> Resource -> Project**
  - Also write down your progress in the same proposal document, referencing the template provided.
  - Create a Team channel in Slack and invite us in.
  - Share the link to this proposal + progress tracking document in your Slack Team channel.
  - We will evaluate, give feedback, and iterate based on your response.
- If needed, arrange to meet with us for consultation during the recess week or in Week 9.

SMU Classification: Restricted

Final Presentation

- Q&A with each team at the front
- Please log in to your app before presenting.
- DO NOT demo about log-in/out, user registration functions
- Open your web pages in inspector mode at 575px (xs) and show JavaScript console
- Prepare to show your github page in browser, if you are using it